COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010/ Data Structures and Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	9/8/26	**DATE SUBMITTED:**	9/8/26
NAME:	Rayven Ferry C. Licas	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 7.1 Sorting Algorithms Pt1

1. PROCEDURE OUTPUT

What procedures did you perform?
Preparation Task:
Create an array of elements with random values. The array must contain 100 elements that are not sorted. Use the created array for each sorting algorithm below. Show the output of the preparation task on table 7-1 in section 6.
Create a header file for the implementation of the different sorting algorithms.
Import this header file into your main.cpp.
A.1. Bubble Sort Technique
Using the bubble sort technique, sorting is done in passes or iteration. Thus at the end of each iteration, the heaviest element is placed at its proper place in the list. In other words, the largest element in the list bubbles up.
General Algorithm

```
template <typename T>
void bubbleSort(T arr[], size_t arrSize){
//Step 1: For i = 0 to N-1 repeat Step 2
for(int i = 0; i < arrSize; i++){
//Step 2: For J = i + 1 to N – I repeat
for(int j = i+1; j < arrSize; j++){
53
//Step 3: if A[J] > A[i]
if(arr[j]>arr[i]){
//Swap A[J] and A[i]
std::swap(arr[j], arr[i]);
}
//[End of Inner for loop]
}
//[End if Outer for loop]
}
//Step 4: Exit
}
```

Include a screenshot of your code and output console along with your observations in table 7-2.
A.2. Selection Sort Algorithm
Selection sort is quite a straightforward sorting technique as the technique only involves finding the smallest element in every pass and placing it in the correct position. Selection sort works efficiently when the list to be sorted is of small size but its performance is affected badly as the list to be sorted grows in size.
Hence we can say that selection sort is not advisable for larger lists of data.

```
template <typename T> int Routine_Smallest(T A[], int K, const int arrSize);
//General Algorithm
//Selection Sort (Array, Size of Array)
template <typename T>
void selectionSort(T arr[], const int N){
int POS, temp, pass=0;
//Step 1: Repeat Steps 2 and 3 for K = 1 to N-1
for(int i = 0; i < N; i++){
//Step 2: Call routine smallest(A, K, N,POS)
POS = Routine_Smallest(arr, i, N);
temp = arr[i];
//Step 3: Swap A[K] with A [POS]
arr[i] = arr[POS];
arr[POS] = temp;
//Count
pass++;
}
//[End of loop]
//Step 4: EXIT
}
//Routine smallest (Array, Current Position, Size of Array)
template <typename T>
int Routine_Smallest(T A[], int K, const int arrSize){
int position, j;
//Step 1: [initialize] set smallestElem = A[K]
T smallestElem = A[K];
//Step 2: [initialize] set POS = K
position = K;
//Step 3: for J = K+1 to N -1,repeat
for(int J=K+1; J < arrSize; J++){
if(A[J] < smallestElem){
smallestElem = A[J];
54
position = J;
}
}
//Step 4: return POS
return position;
}
```

Include a screenshot of your code and output console along with your observations in table 7-3.

A.3. Insertion Sort Algorithm

In the insertion sort technique, we start from the second element and compare it with the first element and put it in a proper place. Then we perform this process for the subsequent elements. We compare each element with all its previous elements and put or insert the element in its proper position. Insertion sort technique is more feasible for arrays with a smaller number of elements. It is also useful for sorting linked lists. Linked lists have a pointer to the next element (in case of a singly linked list) and a pointer to the previous element as well (in case of a doubly linked list). Hence it becomes easier to implement insertion sort for a linked list.

General Algorithm

```
//General Algorithm
//Insertion Sort
template <typename T>
void insertionSort(T arr[], const int N){
int K = 0, J, temp;
//Step 1: Repeat Steps 2 to 5 for K = 1 to N-1
while(K < N){
//Step 2: set temp = A[K]
temp = arr[K];
//Step 3: set J = K – 1
J = K-1;
//Step 4: Repeat while temp <=A[J]
while(temp <= arr[J]){
//set A[J + 1] = A[J]
arr[J+1] = arr[J];
//set J = J – 1
J--;
//[end of inner loop]
}
//Step 5: set A[J + 1] = temp
arr[J+1] = temp;
//[end of loop]
K++;
}
//Step 6: exit
}
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <algorithm>

// Function Declarations
template <typename T> void bubbleSort(T arr[], size_t arrSize);
template <typename T> void selectionSort(T arr[], const int N);
template <typename T> int Routine_Smallest(T A[], int K, const int
arrSize);
template <typename T> void insertionSort(T arr[], const int N);

// Helper function to print array
template <typename T>
```

```cpp
void printArray(T arr[], int size) {
    for(int i = 0; i < size; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << "\n\n";
}

int main() {
    srand(time(0));
    const int SIZE = 100;

    // Preparation Task: Generate array with 100 unsorted random elements
    int originalArray[SIZE];
    for(int i = 0; i < SIZE; i++) {
        originalArray[i] = rand() % 100;
    }

    std::cout << "Table 7-1: Preparation Task - Unsorted Array:\n";
    printArray(originalArray, SIZE);

    // Testing Bubble Sort (Table 7-2)
    int arrBubble[SIZE];
    std::copy(originalArray, originalArray + SIZE, arrBubble);
    bubbleSort(arrBubble, SIZE);
    std::cout << "Table 7-2: Bubble Sort Output:\n";
    printArray(arrBubble, SIZE);

    // Testing Selection Sort (Table 7-3)
    int arrSelection[SIZE];
    std::copy(originalArray, originalArray + SIZE, arrSelection);
    selectionSort(arrSelection, SIZE);
    std::cout << "Table 7-3: Selection Sort Output:\n";
    printArray(arrSelection, SIZE);

    // Testing Insertion Sort (Table 7-4)
    int arrInsertion[SIZE];
    std::copy(originalArray, originalArray + SIZE, arrInsertion);
```

```
    insertionSort(arrInsertion, SIZE);
    std::cout << "Table 7-4: Insertion Sort Output:\n";
    printArray(arrInsertion, SIZE);

    return 0;
}

// A.1. Bubble Sort Technique
template <typename T>
void bubbleSort(T arr[], size_t arrSize){
    //Step 1: For i = 0 to N-1 repeat Step 2
    for(int i = 0; i < arrSize; i++){
        //Step 2: For J = i + 1 to N - I repeat
        for(int j = i+1; j < arrSize; j++){
            //Step 3: if A[J] > A[i]
            if(arr[j]>arr[i]){
                //Swap A[J] and A[i]
                std::swap(arr[j], arr[i]);
            }
            //[End of Inner for loop]
        }
        //[End if Outer for loop]
    }
    //Step 4: Exit
}

// Routine smallest (Array, Current Position, Size of Array)
template <typename T>
int Routine_Smallest(T A[], int K, const int arrSize){
    int position, j;
    //Step 1: [initialize] set smallestElem = A[K]
    T smallestElem = A[K];
    //Step 2: [initialize] set POS = K
    position = K;
    //Step 3: for J = K+1 to N -1,repeat
    for(int J=K+1; J < arrSize; J++){
        if(A[J] < smallestElem){
            smallestElem = A[J];
```

```
            position = J;
        }
    }
    //Step 4: return POS
    return position;
}

// A.2. Selection Sort Algorithm
template <typename T>
void selectionSort(T arr[], const int N){
    int POS, temp, pass=0;
    //Step 1: Repeat Steps 2 and 3 for K = 1 to N-1
    for(int i = 0; i < N; i++){
        //Step 2: Call routine smallest(A, K, N,POS)
        POS = Routine_Smallest(arr, i, N);
        temp = arr[i];
        //Step 3: Swap A[K] with A [POS]
        arr[i] = arr[POS];
        arr[POS] = temp;
        //Count
        pass++;
    }
    //[End of loop]
    //Step 4: EXIT
}

// A.3. Insertion Sort Algorithm
template <typename T>
void insertionSort(T arr[], const int N){
    int K = 0, J, temp;
    //Step 1: Repeat Steps 2 to 5 for K = 1 to N-1
    while(K < N){
        //Step 2: set temp = A[K]
        temp = arr[K];
        //Step 3: set J = K - 1
        J = K-1;
        //Step 4: Repeat while temp <=A[J]
        while(temp <= arr[J]){
```

```
            //set A[J + 1] = A[J]
            arr[J+1] = arr[J];
            //set J = J - 1
            J--;
            //[end of inner loop]
        }
        //Step 5: set A[J + 1] = temp
        arr[J+1] = temp;
        //[end of loop]
        K++;
    }
    //Step 6: exit
}
```

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.